



BITÁCORA GRUPAL DE USO DE IA

PixelForge MathEngine 2D v1.0

Consolidado de interacciones, decisiones y validaciones

Universidad CENFOTEC · Escuela de Fundamentos
FUN-06 · Álgebra Lineal · Sección FCV0 · II Cuatrimestre 2026
Profesor: Steven Gabriel Sánchez Ramírez

Integrantes:
Alejandro Gabriel Medrano Ruiz
Antonio Mora Blotta
José Andrés Picado Corrales
Marcos Antonio Morales Céspedes

30 de julio de 2026

Presentación y criterio de registro

Esta bitácora consolida diez interacciones relevantes documentadas durante el desarrollo de PixelForge MathEngine 2D v1.0. Cada registro conserva los siete campos exigidos: objetivo, prompt, respuesta obtenida, decisión del equipo, validación matemática, modificaciones implementadas y reflexión sobre la utilidad de la IA.

La redacción resume las bitácoras individuales sin cambiar sus decisiones. Los archivos fuente se conservan en docs/bitacoras/. La IA se usó como asistente para proponer estructuras, algoritmos, pruebas, interfaz y documentación; el equipo verificó fórmulas, resultados numéricos y compatibilidad entre módulos.

Índice de interacciones

1. José — representación matricial de objetos
2. José — clonación y formatos de entrada
3. José — Gauss-Jordan, base y subespacios
4. Alejandro — diseño de Transformador
5. Alejandro — validación de matrices
6. Alejandro — casos de prueba
7. Marcos — integración del historial
8. Marcos — composición de transformaciones
9. Antonio — controlador e interfaz
10. Antonio — auditoría de consigna y documentación final

Interacción 1 — José: representación matricial de objetos

Objetivo

Implementar Objeto2D y las fábricas de figuras con una representación matricial $2 \times N$ compatible con las transformaciones posteriores.

Prompt utilizado

- | Se proporcionó a la IA la consigna del módulo y se solicitó crear
- | objetos.py, pruebas, pseudocódigo y bitácora para cuadrados, triángulos y
- | rectángulos, almacenando cada punto como vector.

Respuesta obtenida

La IA propuso una clase con conversión de entradas $2 \times N$, $N \times 2$ y listas de tuplas, métodos de clonación y consulta, y fábricas para las tres figuras. Sugirió `np.copy()` para conservar independencia de memoria.

Decisión tomada por el equipo

Se adoptó $2 \times N$ como formato estándar porque cada columna es un vector y $M \cdot P$ transforma todos los puntos en una operación. Se mantuvo la conversión automática desde $N \times 2$ para facilitar el ingreso de coordenadas.

Validación matemática realizada

Se comprobó que una matriz 2×2 multiplicada por P de tamaño $2 \times N$ produce otra matriz $2 \times N$. Se validaron los cuatro vértices del cuadrado y la conversión de una lista de tres puntos.

Modificaciones implementadas

Se incorporaron validaciones de dimensiones, coordenadas finitas y al menos un punto, además de métodos de presentación y conversión a tuplas.

Reflexión

La propuesta aceleró una base común para el equipo. La revisión del formato fue esencial porque toda transformación, historial y visualización depende de esa convención.

Interacción 2 — José: clonación y formatos de entrada

Objetivo

Confirmar que la clonación fuera independiente y que la normalización de formatos cubriera los casos reales del proyecto.

Prompt utilizado

- | “Revisa si el método clonar de Objeto2D realmente hace copia profunda.
- | Además, verifica entradas (2,N), (N,2), lista de tuplas y formatos inválidos.”

Respuesta obtenida

La IA explicó que `np.copy()` crea un arreglo independiente y revisó cada ruta de conversión. También sugirió validar listas vacías o entradas mal formadas.

Decisión tomada por el equipo

Se mantuvo `np.copy()` y la aceptación de los tres formatos. La validación de objeto vacío se fortaleció en el constructor, mientras las figuras personalizadas exigen al menos tres puntos en el controlador.

Validación matemática realizada

Se clonó un triángulo, se modificó el original y se comprobó que la copia conservó su primera coordenada. También se verificó que transponer $N \times 2$ produce exactamente la matriz $2 \times N$ esperada.

Modificaciones implementadas

Se documentó la diferencia entre copiar una referencia y copiar los datos. El constructor rechaza dimensiones no reconocidas y valores no finitos.

Reflexión

La IA ayudó a identificar un riesgo de aliasing que habría invalidado el historial. La prueba directa fue más importante que aceptar la explicación de la herramienta.

Interacción 3 — José: Gauss-Jordan, base y subespacios

Objetivo

Implementar independencia, base, dimensión, redundancia y análisis de subespacios sin usar funciones directas de `numpy.linalg`.

Prompt utilizado

| “Diseña eliminación de Gauss-Jordan propia para vectores de $\mathbb{R}^2$; úsala para
| rango, independencia, base y redundancia. Explica también cómo decidir si
| $ax + by = c$ es subespacio y cómo tratar el ejemplo $x + y = 10$.”

Respuesta obtenida

La IA propuso contar pivotes para el rango, seleccionar cada vector que aumente ese rango y construir contraejemplos cuando $c \neq 0$. Indicó que $x+y=10$ es una recta afín, no un subespacio.

Decisión tomada por el equipo

Se implementaron Gauss y Gauss-Jordan propios con tolerancia $1e-9$. El análisis de ecuaciones se separó del cierre de la colección finita de vértices para no confundir objetos matemáticos distintos.

Validación matemática realizada

Para seis vectores se obtuvo base $[(1,0),(0,1)]$, dimensión 2 y cuatro índices redundantes. Para $x+y=10$, $(10,0)$ pertenece y $(20,0)$ no; para $x+y=0$, la base $(-1,1)$ tiene dimensión 1.

Modificaciones implementadas

Se añadieron resultados estructurados con cálculos e interpretación geométrica, además de casos para $\mathbb{R}^2$, conjunto vacío, recta homogénea y recta afín.

Reflexión

La IA permitió detectar la ambigüedad del anexo, pero la decisión final surgió de aplicar formalmente los axiomas de subespacio.

Interacción 4 — Alejandro: diseño de Transformador

Objetivo

Diseñar una clase compatible con Objeto2D para traslación, rotación, escalamiento y reflexión, devolviendo el objeto y la matriz usada.

Prompt utilizado

- | “Con objetos almacenados como matrices $2 \times N$, diseñe una clase
- | Transformador en Python con NumPy. Incluya las cuatro operaciones y no use
- | funciones directas prohibidas.”

Respuesta obtenida

La IA propuso multiplicación matricial para las operaciones lineales y coordenadas homogéneas para la traslación, que no es lineal en $\mathbb{R}^2$.

Decisión tomada por el equipo

Se aceptó la interfaz (objeto_transformado, matriz) y se decidió crear siempre un objeto nuevo. La traslación usa puntos $(x,y,1)$ y una matriz 3×3 .

Validación matemática realizada

Se confirmó que $M(2 \times 2) \cdot P(2 \times N)$ transforma todas las columnas y que la matriz homogénea produce $(x+dx, y+dy, 1)$.

Modificaciones implementadas

Se agregaron validaciones de tipo y números finitos, registro de última operación y un auxiliar para construir el objeto resultante sin alterar el original.

Reflexión

La IA aportó una estructura clara, pero el equipo verificó que la representación homogénea fuera coherente con el formato $2 \times N$ elegido por el módulo central.

Interacción 5 — Alejandro: validación de matrices

Objetivo

Revisar los signos, dimensiones y efectos geométricos de todas las matrices.

Prompt utilizado

- | “Verifique las matrices de rotación, escala, reflexión en X, Y y $y=x$, y
- | traslación homogénea. Explique el efecto de cada una.”

Respuesta obtenida

La IA confirmó las matrices estándar y explicó su efecto: cambio de orientación alrededor del origen, modificación de tamaño, inversión respecto a un eje e incremento común de coordenadas.

Decisión tomada por el equipo

Se mantuvieron las matrices y se decidió recibir ángulos en grados para coincidir con los ejemplos de 45° y 30° de la consigna.

Validación matemática realizada

Se verificaron manualmente $(2,0) \rightarrow (1.414,1.414)$ a 45° , $(2,2) \rightarrow (4,4)$ al escalar por 2, $(2,3) \rightarrow (2,-3)$ al reflejar en X, $(-2,3)$ en Y, $(3,2)$ en $y=x$ y $(2,2) \rightarrow (5,1)$ al trasladar por $(3,-1)$.

Modificaciones implementadas

`rotar()` convierte grados a radianes y `escalar()` admite un único factor para escala uniforme.

Reflexión

La revisión mostró que una respuesta plausible no basta: un solo signo incorrecto cambia el sentido de rotación o el eje reflejado.

Interacción 6 — Alejandro: casos de prueba

Objetivo

Crear evidencia ejecutable para las cuatro transformaciones y la secuencia solicitada.

Prompt utilizado

| “Genere pruebas para traslación, rotación 45°, escala por 2, tres reflexiones
| y la secuencia rotar 30°, escalar 1.5 y trasladar. Muestre matrices y
| coordenadas.”

Respuesta obtenida

La IA sugirió `np.allclose()` para comparar resultados trigonométricos y un caso de error para ejes de reflexión no soportados.

Decisión tomada por el equipo

Se aceptó `allclose` únicamente como validador; no resuelve el algoritmo. También se construyó la matriz esperada de la secuencia de forma independiente.

Validación matemática realizada

La composición se calculó como $C=T \cdot S \cdot R$. Sus puntos coincidieron con aplicar las operaciones una por una y el objeto original permaneció sin cambios.

Modificaciones implementadas

Las pruebas imprimen figura original, matriz, figura transformada y coordenadas. Se añadió validación del error para eje="z".

Reflexión

La IA aceleró la cobertura, pero los resultados esperados se revisaron manualmente para evitar pruebas que solo repitieran el mismo error del código.

Interacción 7 — Marcos: integración del historial

Objetivo

Conectar las tuplas devueltas por Transformador con el historial sin duplicar lógica ni modificar módulos ya probados.

Prompt utilizado

| “¿Cómo conectar (objeto_transformado, matriz_utilizada) con
| HistorialTransformaciones evitando repetir el registro en cada operación?”

Respuesta obtenida

La IA recomendó una función `aplicar_y_registrar()` que desempaqueta el resultado, crea el registro y devuelve el nuevo objeto para continuar el flujo.

Decisión tomada por el equipo

Se adoptó el auxiliar porque conserva la interfaz de Transformador y centraliza la lógica de historial.

Validación matemática realizada

Se aplicaron rotación y escala consecutivas. El estado posterior del primer registro coincidió con el estado anterior del segundo, y las matrices guardadas fueron las mismas que produjo el transformador.

Modificaciones implementadas

Se creó `aplicar_y_registrar()` y se reutilizó tanto en la secuencia oficial como en el controlador de interfaz y la prueba integrada.

Reflexión

La propuesta favoreció integración modular. Las copias independientes de estados se verificaron para impedir que una modificación posterior alterara el historial.

Interacción 8 — Marcos: composición de transformaciones

Objetivo

Implementar rotación 30° , escala 1.5 y traslación (2,1), mostrando matrices individuales y composición.

Prompt utilizado

| “Explique cómo componer las matrices en ese orden cuando rotación y escala
| son 2×2 y la traslación es homogénea 3×3 .”

Respuesta obtenida

La IA explicó que, con vectores columna, la matriz más cercana se aplica primero; por eso $C=T \cdot S \cdot R$. Recomendó ampliar las matrices 2×2 a 3×3 .

Decisión tomada por el equipo

Se implementó la conversión homogénea y se multiplica cada nueva matriz por la izquierda al recorrer el historial.

Validación matemática realizada

La matriz obtenida fue:

```
[ [1.299038, -0.750000, 2.000000],  
  [0.750000, 1.299038, 1.000000],  
  [0.000000, 0.000000, 1.000000] ]
```

Aplicarla al objeto inicial reconstruyó los mismos puntos que la secuencia.

Modificaciones implementadas

Se añadieron `_matriz_homogenea()`, `matriz_compuesta()`, `reconstruir_estado_final()` y `aplicar_transformaciones_consecutivas()`.

Reflexión

La interacción aclaró el orden, una fuente frecuente de errores. La equivalencia numérica transformó la explicación en evidencia verificable.

Interacción 9 — Antonio: controlador e interfaz

Objetivo

Integrar el repositorio en una interfaz visual sin duplicar algoritmos matemáticos y mostrar todos los resultados solicitados.

Prompt utilizado

| “Explora y entiende el codebase y la consigna. Implementa una interfaz gráfica
| conectada a la lógica existente.”

Respuesta obtenida

La IA propuso separar un controlador probado de la ventana Tkinter y organizar la vista en controles, gráficos comparativos y pestañas de resultados.

Decisión tomada por el equipo

Se adoptó Motor2DController para coordinar objetos, transformaciones, historial y análisis. Los gráficos comparten límites para que la comparación no resulte engañosa.

Validación matemática realizada

La secuencia oficial produjo las mismas coordenadas y matriz T·S·R que las pruebas. Se verificó preservación del original, reinicio, figuras personalizadas y análisis de subespacios.

Modificaciones implementadas

Se creó src/interfaz.py, se añadieron pruebas del controlador, exportación PNG, cálculo $M \cdot P = P'$, interpretación geométrica y modo --demo.

Reflexión

La IA fue útil para la organización visual. Separar controlador y vista permitió validar el flujo aun en un entorno sin pantalla.

Interacción 10 — Antonio: auditoría y documentación final

Objetivo

Comparar la consigna PDF con el DOCX de seguimiento y completar la documentación faltante con evidencia real del proyecto.

Prompt utilizado

| “En este repo está la consigna en PDF completa. Falta la documentación que
| solicitan y está marcada como no completada en el DOCX. Primero revisa contra
| la consigna —fuente de verdad— que el DOCX no dejó nada por fuera; luego ayuda
| a completar la documentación.”

Respuesta obtenida

La IA extrajo las trece páginas de la consigna, leyó la estructura del DOCX, revisó código y documentos, ejecutó pruebas y detectó cinco aspectos no explícitos en el seguimiento: cálculos realizados, interpretación geométrica, mejoras, mínimo de casos y retroalimentación docente.

Decisión tomada por el equipo

Se decidió integrar revisión bibliográfica e informe técnico en un solo PDF, conservar fuentes editables Markdown, consolidar diez interacciones y crear una matriz de trazabilidad. No se inventó retroalimentación docente ausente.

Validación matemática realizada

Se ejecutaron 14 casos unittest y 15 escenarios con aserciones, todos sin errores. Se reprodujo $C=T \cdot S \cdot R$, sus puntos finales, la base y dimensión, y el contraejemplo para $x+y=10$.

Modificaciones implementadas

Se generaron Informe_Tecnico.pdf, Bitacora_IA_Grupal.pdf, sus fuentes, figuras reproducibles y MATRIZ_CUMPLIMIENTO.md; además se actualizó el DOCX de seguimiento.

Reflexión

La IA aportó valor al cruzar requisitos con evidencia concreta. La declaración explícita de pendientes externos es más responsable que marcar cumplimiento sin respaldo.

Cierre de la bitácora

Las diez interacciones superan el mínimo recomendado de cinco y cubren todas las etapas principales: representación, análisis, transformaciones, integración, interfaz, pruebas y documentación.

El patrón común fue:

1. formular una necesidad concreta;
2. recibir una propuesta de la IA;
3. evaluar su compatibilidad con la consigna y los módulos existentes;
4. validar matemáticamente o mediante pruebas independientes;
5. modificar lo necesario;
6. documentar la decisión y sus límites.

La IA no sustituyó la comprensión del equipo. Los ejemplos de mayor riesgo —el orden de composición, los signos de rotación, la diferencia entre recta afín y subespacio, y el significado de redundancia— se revisaron con cálculos explícitos. Los resultados quedaron reproducibles en pruebas, informes y código.

Archivos fuente

- docs/bitacoras/bitacora_jose.txt
- docs/bitacoras/bitacora_alejandro.txt
- docs/bitacoras/bitacora_marcos.txt
- docs/bitacoras/bitacora_antonio.txt

Esta versión grupal organiza el material para la entrega y no elimina las bitácoras individuales que permiten rastrear el origen de cada interacción.